*BỘ GIÁO DỤC VÀ ĐÀO TẠO*

TRƯỜNG ĐẠI HỌC KIẾN TRÚC ĐÀ NẴNG

KHOA CÔNG NGHỆ THÔNG TIN

*BÁO CÁO ĐỒ ÁN TRÍ TUỆ NHÂN TẠO*

23CT1 - AIP20201

Tên đề tài:

# Xây dựng hệ thống nhận diện vật thể và dịch nội dung hình ảnh đa ngôn ngữ

Sinh viên thực hiện : *Phạm Anh Tú*

: *Lê Thịnh*

Lớp : *CNTT 23CT1*

Giảng viên hướng dẫn : *TS. Đỗ Phúc Hảo*

Năm học : *2026 – 2027*

*Đà Nẵng 2026 - 2027*

# Lời Cam Đoan

Nhóm em xin cam đoan đồ án này là công trình do chính tôi thực hiện dưới sự hướng dẫn của TS. Đỗ Phúc Hảo. Các nội dung nghiên cứu, số liệu và kết quả trình bày trong đồ án là trung thực và chưa từng được công bố trong bất kỳ công trình nào khác.

Mọi số liệu, hình ảnh, mã nguồn và kết quả của tác giả khác được sử dụng trong đồ án đều được trích dẫn nguồn gốc rõ ràng trong phần tài liệu tham khảo. Toàn bộ phần mềm và thư viện được dùng đều có giấy phép cho phép sử dụng. Những phần có sử dụng công cụ trí tuệ nhân tạo hỗ trợ đã được nêu rõ trong báo cáo.

Nếu phát hiện có bất kỳ sự gian lận nào, nhóm em xin chịu hoàn toàn trách nhiệm trước Khoa và Nhà trường.

Đà Nẵng, ngày … tháng … năm…

**Sinh viên thực hiện**

Phạm Anh Tú | Lê Thịnh

# LỜI CẢM ƠN

Nhóm em xin gửi lời cảm ơn chân thành đến TS. Đỗ Phúc Hảo, người đã tận tình định hướng đề tài, góp ý và theo sát tiến độ của nhóm em trong suốt quá trình thực hiện đồ án "Xây dựng hệ thống nhận diện vật thể và dịch nội dung hình ảnh đa ngôn ngữ". Nhóm em cũng xin cảm ơn quý thầy cô Khoa Công nghệ Thông tin, Trường Đại học Kiến trúc Đà Nẵng đã trang bị cho nhóm em nền tảng kiến thức để hoàn thành công việc này, đặc biệt là những kiến thức liên quan đến lĩnh vực Trí tuệ nhân tạo, giúp nhóm em có cơ sở để nghiên cứu và triển khai đề tài. Cuối cùng, nhóm em xin cảm ơn gia đình và bạn bè đã luôn động viên nhóm trong thời gian học tập.

Em xin trân trọng cảm ơn!

# MỤC LỤC

**CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI** . . . . . 1
1.1. Lý do chọn đề tài . . . . . 1
1.2. Mục tiêu của đề tài . . . . . 1
1.3. Đối tượng và phạm vi nghiên cứu . . . . . 2
1.4. Phương pháp thực hiện . . . . . 3
1.5. Cấu trúc của đề tài . . . . . 3
**Tiểu kết chương 1** . . . . . 4
**CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ** . . . . . 5
2.1. Trí tuệ nhân tạo và thị giác máy tính . . . . . 5
2.2. Nhận dạng ký tự quang học OCR . . . . . 5
2.3. PaddleOCR . . . . . 6
2.4. Nhận diện vật thể với YOLO11 . . . . . 6
2.5. Phân đoạn ngữ nghĩa với SegFormer . . . . . 7
2.6. Dịch máy đa ngôn ngữ . . . . . 7
2.7. MongoDB và lưu trữ hình ảnh . . . . . 8
**Tiểu kết chương 2** . . . . . 8
**CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG** . . . . . 9
3.1. Mô tả bài toán . . . . . 9
3.2. Yêu cầu chức năng . . . . . 9
3.3. Yêu cầu phi chức năng . . . . . 10
3.4. Kiến trúc tổng thể . . . . . 10
3.5. Sơ đồ Use Case tổng quát . . . . . 12
3.6. Sơ đồ luồng tổng thể . . . . . 13
3.7. Sơ đồ tuần tự đăng nhập . . . . . 14
3.8. Sơ đồ tuần tự tải ảnh lên . . . . . 16
3.9. Sơ đồ tuần tự nhận diện vật thể . . . . . 17
3.10. Sơ đồ tuần tự OCR và dịch thuật . . . . . 18
3.11. Sơ đồ hoạt động phân tích ảnh đa mô hình . . . . . 20
**Tiểu kết chương 3** . . . . . 21
**CHƯƠNG 4. KIỂM THỬ CÁC CHỨC NĂNG** . . . . . 22
4.1. Môi trường kiểm thử . . . . . 22
4.2. Kiểm thử chức năng . . . . . 22
4.3. Kiểm thử OCR . . . . . 23

4.4. Kiểm thử dịch thuật . . . . . 24
4.5. Kiểm thử hệ thống đa mô hình . . . . . 24
4.7. Đánh giá kết quả . . . . . 25
**Tiểu kết chương 4** . . . . . 26
**CHƯƠNG 5. HƯỚNG PHÁT TRIỂN VÀ KẾT LUẬN** . . . . . 27
5.1. Tổng kết kết quả đạt được . . . . . 27
5.2. Những hạn chế của hệ thống . . . . . 28
5.3. Hướng phát triển trong tương lai . . . . . 29
**KẾT LUẬN** . . . . . 30

Link github: Thinh030505/AI_FINAL: Phase3

# DANH MỤC HÌNH VẼ / SƠ ĐỒ

Hình 3.1 Giao diện Trang Chủ . . . . . 11
Hình 3.2 Sơ đồ Use Case tổng quát . . . . . 12
Hình 3.3 Sơ đồ luồng tổng thể hệ thống . . . . . 14
Hình 3.4 Sơ đồ tuần tự đăng nhập . . . . . 15
Hình 3.5 Giao diện đăng nhập hệ thống . . . . . 15
Hình 3.6 Sơ đồ tuần tự Upload ảnh . . . . . 16
Hình 3.7 Tuần tự nhận diện vật thể . . . . . 17
Hình 3.8 Giao diện hình ảnh bằng Camera . . . . . 18
Hình 3.9 Tuần tự OCR và dịch thuật . . . . . 18
Hình 3.10 Giao diện dạng dạng văn bản và dịch thuật . . . . . 19
Hình 3.11 Hoạt động phân tích ảnh đa mô hình . . . . . 20

# MỞ ĐẦU

Hình ảnh chụp từ điện thoại hoặc webcam có thể chứa văn bản, vật thể, con người và bối cảnh khác nhau. Nếu mỗi loại thông tin phải được xử lý bằng một chức năng riêng, người dùng cần chuyển qua lại giữa nhiều màn hình và kết quả không được tổng hợp trong một bản ghi thống nhất. Đề tài xây dựng một ứng dụng web cho phép tải ảnh hoặc lấy khung hình từ camera, sau đó dùng các dịch vụ AI để trích xuất văn bản, phát hiện vật thể, phân đoạn vùng ảnh và tổng hợp thông tin.

Repository thể hiện hệ thống gồm ba ứng dụng chạy độc lập: giao diện React/TypeScript, backend NestJS và dịch vụ AI FastAPI/Python. Backend tiếp nhận yêu cầu từ giao diện, xác thực người dùng, làm việc với MongoDB và dịch vụ lưu trữ Cloudinary, sau đó gọi AI service khi cần phân tích. AI service cung cấp các endpoint OCR, dịch thuật, phát hiện vật thể, phân đoạn ngữ nghĩa, phân tích ảnh kết hợp và phân tích khung hình camera.

Phạm vi báo cáo tập trung vào chức năng có thể truy vết trong source: luồng upload ảnh, tài khoản và JWT, OCR tiếng Việt, dịch văn bản, YOLO, SegFormer, phân tích ảnh đa mô hình, camera trực tiếp, lịch sử dữ liệu và các script kiểm thử/huấn luyện. Báo cáo không mặc định mọi mô hình tùy chọn đều được cài đặt, không khẳng định các endpoint đã hoạt động ở một môi trường cụ thể, và không đưa ra độ chính xác thực nghiệm nếu không có kết quả kiểm thử thực tế được xác nhận.

# CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

## 1.1. Lý do chọn đề tài

Trong nhiều tình huống thực tế, người dùng cần khai thác đồng thời nội dung chữ và nội dung hình ảnh. Một ảnh hóa đơn cần trích xuất chữ; một ảnh đường phố cần xác định các vật thể và khu vực; một khung hình camera có thể cần nhận dạng đối tượng liên tục. Mã nguồn dự án thể hiện nhu cầu kết hợp các thao tác này trong một ứng dụng: frontend có các trang OCR, phát hiện vật thể, phân tích ảnh, camera và lịch sử; backend có các module tương ứng; AI service cung cấp các mô hình xử lý độc lập và pipeline hợp nhất.

Lý do lựa chọn đề tài vì vậy xuất phát từ bài toán tích hợp: tạo một giao diện duy nhất để người dùng đưa ảnh vào, gọi đúng năng lực AI, xem kết quả có cấu trúc và truy xuất lại lịch sử. Dự án cũng đặt trọng tâm vào ảnh có văn bản tiếng Việt thông qua cấu hình PaddleOCR lang=’vi’, chuẩn hóa Unicode và xử lý nhiều lượt nhận dạng; đồng thời mở rộng từ OCR sang nhận biết nội dung tổng thể của ảnh.

## 1.2. Mục tiêu của đề tài

Xây dựng hệ thống web đa dịch vụ hỗ trợ nhận dạng văn bản và phân tích hình ảnh bằng nhiều mô hình, có cơ chế lưu kết quả theo tài khoản người dùng.

| STT | Mục tiêu | Kết quả mong muốn |
|---|---|---|
| 1 | Xây dựng giao diện web | React 18 + Vite + TypeScript + Tailwind CSS |
| 2 | Nhận dạng văn bản | PaddleOCR xử lý văn bản tiếng Việt và tiếng Anh |
| 3 | Dịch nội dung | Dịch Việt ↔ Anh và có cơ chế fallback |
| 4 | Nhận diện vật thể | YOLO11n trả về nhãn, confidence và bounding box |
| 5 | Phân đoạn ảnh | SegFormer xác định các vùng ngữ nghĩa |
| 6 | Hợp nhất AI | Kết hợp OCR, YOLO11, SegFormer và suy luận bối cảnh |

### 1.3. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu:

- Quy trình xử lý ảnh đầu vào trong một ứng dụng web.
- Việc tích hợp các mô hình OCR, object detection, semantic segmentation và dịch thuật.
- Giao tiếp REST giữa frontend, backend và AI service.
- Lưu dữ liệu người dùng, ảnh và kết quả xử lý bằng Mongoose/MongoDB.
- Cách trình bày kết quả AI trong giao diện web và trong camera trực tiếp.

Phạm vi chức năng thể hiện trong mã nguồn:

- Hệ thống hiện có các trang đăng nhập/đăng ký, dashboard, OCR, phát hiện vật thể, phân tích ảnh, camera và lịch sử. Backend định nghĩa module cho xác thực, người dùng, ảnh, OCR, dịch thuật, phát hiện vật thể, phân tích ảnh, camera, dịch thuật, phát hiện vật thể, phân đoạn, phân tích ảnh.

Phạm vi không khẳng định:

- Không khẳng định dịch thuật chỉ giới hạn Việt–Anh: service nhận mã ngôn ngữ từ request, nhưng logic tự nhận dạng trong code chỉ là heuristic đơn giản giữa tiếng Việt và tiếng Anh.

- Không đưa ra tỷ lệ chính xác, thời gian xử lý hay kết quả "đạt 100%" như kết luận thực nghiệm nếu không chạy và xác nhận bộ kiểm thử trên đúng môi trường.

## 1.4. Phương pháp thực hiện

Quy trình thể hiện qua tổ chức source có thể mô tả thành các bước:

1. Phân tích yêu cầu qua các màn hình và API: xác định các luồng đăng nhập, upload, OCR, dịch, phát hiện, phân tích tổng hợp, camera và lịch sử.
2. Tách hệ thống thành dịch vụ: giao diện web, backend nghiệp vụ và AI service độc lập.
3. Thiết kế hợp đồng dữ liệu: dùng DTO/validation ở NestJS, Pydantic model ở FastAPI và các TypeScript type ở frontend.
4. Tích hợp mô hình: đóng gói thao tác gọi PaddleOCR, YOLO, SegFormer và translator thành service, sau đó kết nối qua API.
5. Lưu kết quả: lưu metadata ảnh và các kết quả nghiệp vụ trong các Mongoose schema.
6. Hiển thị và truy xuất: gọi API từ frontend, trực quan hóa kết quả và tải lịch sử.
7. Kiểm thử/đánh giá: repository có các script gọi API, script benchmark, script huấn luyện và script đánh giá detector.

## 1.5. Cấu trúc của đề tài

- Chương 1 Tổng quan đề tài.
- Chương 2 Cơ sở lí thuyết.
- Chương 3 Phân tích và thiết kế
- Chương 4 Kiểm thử chức năng
- Chương 5 Hướng phát triển và kết luận

**Tiểu kết chương 1**

Đề tài hướng tới một ứng dụng phân tích ảnh đa chức năng, trong đó frontend là điểm tương tác, NestJS quản lý nghiệp vụ và FastAPI chạy các tác vụ AI. Phạm vi nghiên cứu bám vào chức năng đã được nối giữa các tầng và không lấy tên mô hình hoặc kết quả trong tài liệu mô tả làm bằng chứng thay thế cho source.

# CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ

## 2.1. Trí tuệ nhân tạo và thị giác máy tính

Trong dự án, thị giác máy tính được thể hiện qua các tác vụ xử lý ảnh cụ thể: phát hiện vật thể trả về nhãn và bounding box; phân đoạn trả về các lớp vùng ảnh cùng tỷ lệ diện tích; OCR trả văn bản và vị trí dòng; camera trực tiếp kết hợp phát hiện, phân cấp, tracking và hiển thị.

Pipeline phân tích ảnh không chỉ gọi một mô hình. Endpoint /image-analysis chuẩn bị ảnh dùng chung, đánh giá chất lượng, chọn cấu hình thực thi, chạy YOLO và SegFormer, có thể chạy OCR tùy theo mode và tín hiệu văn bản, thực hiện phân tích người/vật nhỏ, suy luận cảnh, gọi vision-language service và hợp nhất kết quả.

Trạng thái của từng module và thời gian xử lý được đưa vào đầu ra. Vì vậy, thuật ngữ "đa mô hình" trong báo cáo chỉ sự kết hợp các kết quả này trong cùng một pipeline; không hàm ý các model luôn chạy song song hoặc luôn thành công trong mọi request.

## 2.2. Nhận dạng ký tự quang học OCR

OCR trong hệ thống nhận ảnh và tạo ra văn bản có cấu trúc. Kết quả của service gồm fullText, danh sách dòng, độ tin cậy, tọa độ đa giác, thời gian xử lý, văn bản trước hậu xử lý và một số trường debug. Backend lưu lại kết quả cùng userId và imageId.

Trước nhận dạng, ảnh có thể được tải và tiền xử lý. Endpoint OCR độc lập dùng chế độ adaptive cho lượt đầu; nếu confidence ở mức cần kiểm tra thì tạo lượt xử lý tương phản để so sánh. Dòng OCR được sắp thứ tự theo vị trí hình học, sau đó hậu xử lý Unicode.

## 2.3. PaddleOCR

OCRService khởi tạo PaddleOCR với lang='vi'. Service hỗ trợ phân tích hai dạng kết quả khác nhau của PaddleOCR: cấu trúc dictionary có rec_texts/rec_scores/rec_polys, và cấu trúc list/tuple kiểu box-text-confidence.

Các bước xử lý được thể hiện trong code:

1. Gọi OCR trên ảnh đã tiền xử lý adaptive.
2. Chuyển kết quả thành danh sách TextLine, mỗi dòng có text, confidence và box.
3. Sắp xếp dòng theo tâm trục Y, nhóm các dòng gần nhau và sắp theo X trong mỗi nhóm.
4. Nếu có dòng nhưng confidence trung bình dưới 0.90 hoặc confidence thấp nhất dưới 0.75, service có thể chạy lượt hai bằng ảnh tăng tương phản.
5. So sánh số dòng, confidence và số ký tự ngoài ASCII giữa hai lượt để chọn kết quả.
6. Chuẩn hóa và hậu xử lý văn bản, trả fullText, confidence trung bình/tối thiểu và thông tin debug.

Các ngưỡng trên là điều kiện trong implementation, không phải kết quả kiểm chứng độ chính xác tổng thể. Trong pipeline phân tích ảnh, OCR có thể bị bỏ qua theo routing hoặc được bật bằng forceOcr; giao diện OCR riêng gọi endpoint OCR chuyên biệt.

## 2.4. Nhận diện vật thể với YOLO11

Object detection độc lập nhận ảnh, chạy Ultralytics YOLO và trả danh sách label, classId, confidence, bounding box, kích thước ảnh, model và thời gian xử lý. Kết quả được lưu trong MongoDB và dùng để dựng overlay ở giao diện.

Service tìm trọng số theo thứ tự: ai-service/models/best.pt, sau đó ai-service/yolo11n.pt, và cuối cùng tên model yolo11n.pt để Ultralytics tải/khởi tạo theo môi trường. Tên model được đánh dấu trong service theo nhánh chọn

trọng số. Nếu YOLO_RUNTIME cho phép kiểm tra ONNX và tồn tại tệp .onnx tương ứng cùng thư viện onnxruntime, service thử dùng ONNX; nếu không thì dùng PyTorch. Thiết bị được chọn theo trạng thái CUDA.

Đối với camera thời gian thực, RealTimeAIRouter có bộ model riêng: YOLO tổng quát, YOLO pose và tùy chọn YOLO-World nếu file trọng số tương ứng có sẵn. Pipeline này còn có quy tắc lọc một số lớp theo confidence/diện tích, xác định vùng người, hợp nhất detections, xây dựng quan hệ cha-con và cập nhật

### 2.5. Phân đoạn ngữ nghĩa với SegFormer

SegmentationService nạp model nvidia/segformer-b0-finetuned-ade-512-512 thông qua Transformers và chạy trên CUDA nếu có, nếu không thì dùng CPU. Kết quả logits được nội suy về kích thước ảnh ban đầu, lấy lớp có giá trị cao nhất theo pixel, đếm số pixel từng lớp và gom một số nhãn tương đương thành nhãn chuẩn như water/sea, ground, tree/foliage.

API hiện trả các vùng dưới dạng thống kê: nhãn gốc, nhãn đã chuẩn hóa, số pixel và phần trăm diện tích. Các vùng nhỏ hơn 1% diện tích ảnh bị loại khỏi danh sách trả về. Output của endpoint trong code không phải ảnh mask màu cho từng pixel; frontend hiển thị danh sách/vùng theo dữ liệu phần trăm và bảng màu.

### 2.6. Dịch máy đa ngôn ngữ

AI service dùng deep-translator với GoogleTranslator làm lần thử đầu và MyMemoryTranslator làm dự phòng. Văn bản được chuẩn hóa Unicode NFC, chia theo dòng để giữ cấu trúc xuống dòng và dịch từng dòng. Khi sourceLanguage=’auto’, service dùng dấu tiếng Việt để chọn vi, nếu không tìm thấy thì chọn en; đây là heuristic theo code, không phải bộ nhận dạng ngôn ngữ tổng quát.

Backend yêu cầu dịch qua NestJS, ghi lịch sử dịch theo người dùng và trả text nguồn, text dịch, ngôn ngữ nguồn/đích cùng thời gian xử lý. Frontend OCR

có thể lấy fullText từ kết quả OCR làm đầu vào dịch. Nếu cả hai dịch vụ ngoài không trả kết quả, implementation hiện giữ nguyên dòng nguồn làm fallback; vì vậy khi đánh giá cần phân biệt request thành công về mặt API với nội dung thực sự đã được dịch.

## 2.7. MongoDB và lưu trữ hình ảnh

Backend sử dụng Mongoose và các schema cho User, Image, OCRResult, Translation, ObjectDetection và ImageAnalysis. Các bản ghi kết quả gắn với userId; nhiều schema lưu imageId để liên kết ảnh gốc. Lịch sử thường được truy vấn theo userId, sắp xếp mới nhất trước.

Ảnh được kiểm tra MIME type và giới hạn dung lượng cấu hình trước khi upload. ImagesService gọi Cloudinary, sau đó lưu URL, public ID, tên tệp, MIME type, kích thước, kích thước pixel, nguồn và thời điểm chụp vào MongoDB.

## Tiểu kết chương 2

Các công nghệ trong chương này được xác định từ import, cấu hình, service và API thực tế: PaddleOCR cho OCR, YOLO/Ultralytics cho detection, Transformers SegFormer cho segmentation, deep-translator cho dịch, Gemini tùy cấu hình cho vision-language và MongoDB/Mongoose cùng Cloudinary cho lưu trữ. Kết quả do các model tạo ra có thể phụ thuộc trọng số, môi trường, cấu hình và dịch vụ ngoài.

# CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

## 3.1. Mô tả bài toán

Người dùng cần chọn một ảnh hoặc lấy ảnh từ camera, gửi yêu cầu phân tích, nhận kết quả và tra cứu lại. Bài toán có các yêu cầu phối hợp:

Với ảnh tĩnh, quy trình thường là upload ảnh trước rồi gửi imageId tới endpoint nghiệp vụ. Backend xác thực quyền sở hữu ảnh, lấy URL đã lưu, gọi AI service và ghi bản ghi kết quả. Với camera trực tiếp, từng frame được gửi lên endpoint smart-camera để xử lý tức thời; người dùng có thể chụp ảnh tĩnh để chuyển sang quy trình upload và phân tích ảnh lưu trữ.

## 3.2. Yêu cầu chức năng

| Nhóm | Chức năng | Mô tả |
|---|---|---|
| Tài khoản | Đăng ký/Đăng nhập | Tạo tài khoản, xác thực và cấp JWT. |
| Ảnh | Upload/Quản lý | Tải ảnh, xem metadata, xóa ảnh. |
| OCR | Nhận dạng chữ | Trả về fullText, dòng, box và confidence. |
| Translation | Dịch văn bản | Nhận diện nguồn, chọn ngôn ngữ đích và lưu lịch sử. |
| Detection | Nhận diện vật thể | YOLO11n trả label, classId, confidence, box. |
| Understanding | Hiểu ảnh | Kết hợp OCR, YOLO11, SegFormer và Vision Synthesizer. |
| History | Lịch sử | Xem lại ảnh, OCR, translation, object detection và AI understanding. |
| Security | Phân quyền | Chỉ cho phép chủ sở hữu truy cập dữ liệu của mình. |

### 3.3. Yêu cầu phi chức năng

1. Tách biệt trách nhiệm: giao diện xử lý trình bày; NestJS xử lý tài khoản, dữ liệu và điều phối; FastAPI tập trung vào inference.
2. Kiểm tra đầu vào: NestJS bật ValidationPipe với whitelist, transform và forbidNonWhitelisted; upload kiểm tra loại MIME và kích thước.
3. Phân quyền dữ liệu: các route ảnh tĩnh và lịch sử phân tích dùng JWT; service kiểm tra chủ sở hữu dựa trên userId.
4. Khả năng phục hồi ở mức module: pipeline ghi trạng thái từng module như success, failed, skipped, fallback; một số module có thể lỗi mà vẫn tạo kết quả một phần.
5. Hiệu năng: ảnh được tái sử dụng trong ProcessedImageBundle; YOLO và SegFormer được chạy song song trong pipeline ảnh; có giới hạn concurrency phân tích và cache kết quả.
6. Tương thích thiết bị: service kiểm tra CUDA cho một số model và dùng CPU khi không có CUDA.

### 3.4. Kiến trúc tổng thể

1. Người dùng thao tác trên Frontend React + TypeScript.
2. Frontend gọi Backend NestJS bằng REST kèm JWT.
3. Backend ghi/đọc dữ liệu MongoDB qua Mongoose, upload/xóa ảnh trên Cloudinary và gọi AI Service FastAPI bằng HTTP nội bộ.

➤ ***Tầng giao diện***

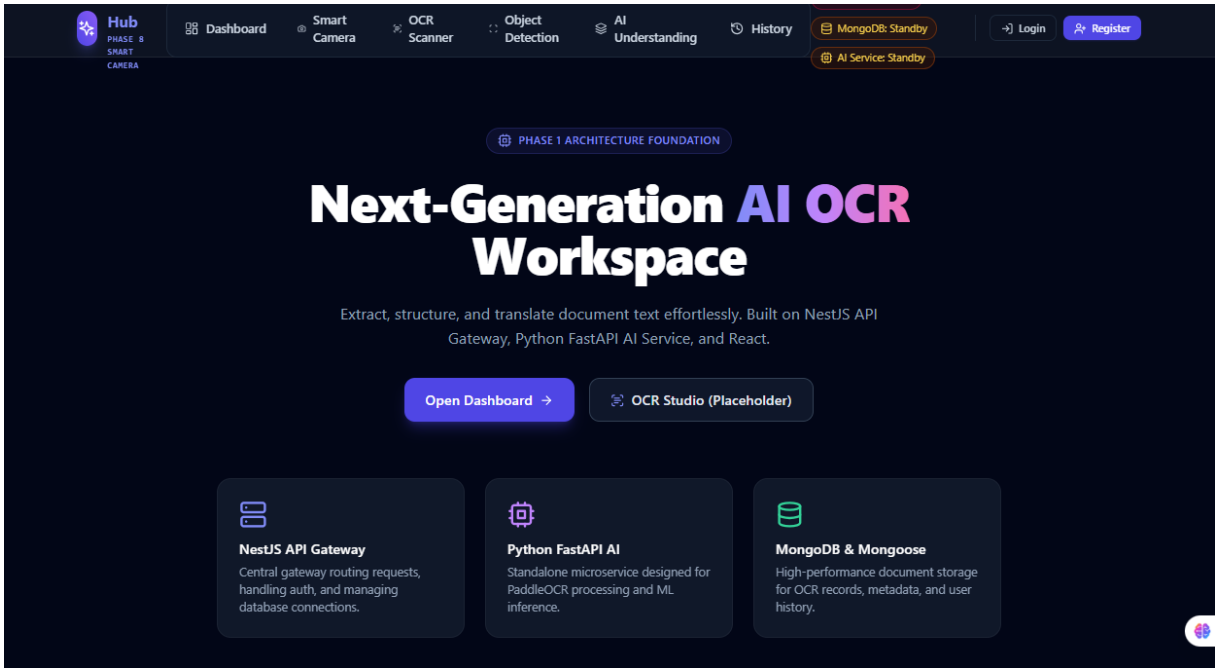
- Frontend sử dụng React, TypeScript, Vite, React Router và Axios. AuthContext quản lý thông tin đăng nhập và token ở trình duyệt; ProtectedRoute yêu cầu xác thực cho các trang dashboard, OCR, detection, phân tích ảnh, camera và history. Các service frontend gọi API qua một apiClient dùng VITE_API_URL.

➤ ***Tầng backend***

- NestJS đặt tiền tố /api, khai báo các module nghiệp vụ và kết nối MongoDB qua Mongoose. Backend đảm nhận xác thực, kiểm tra DTO, quyền sở hữu ảnh, upload Cloudinary, gọi FastAPI và lưu lịch sử.

➤ ***Tầng AI***

- FastAPI đăng ký router OCR, translation, object detection, segmentation, image analysis, feedback và smart camera. AI service thường nhận URL ảnh từ backend, tải và giải mã ảnh rồi chạy model.




Hình 3.1. Giao diện Trang Chủ

### 3.5. Sơ đồ Use Case tổng quát




Hình 3.2. Sơ đồ Use Case tổng quát

Các ca sử dụng của người dùng

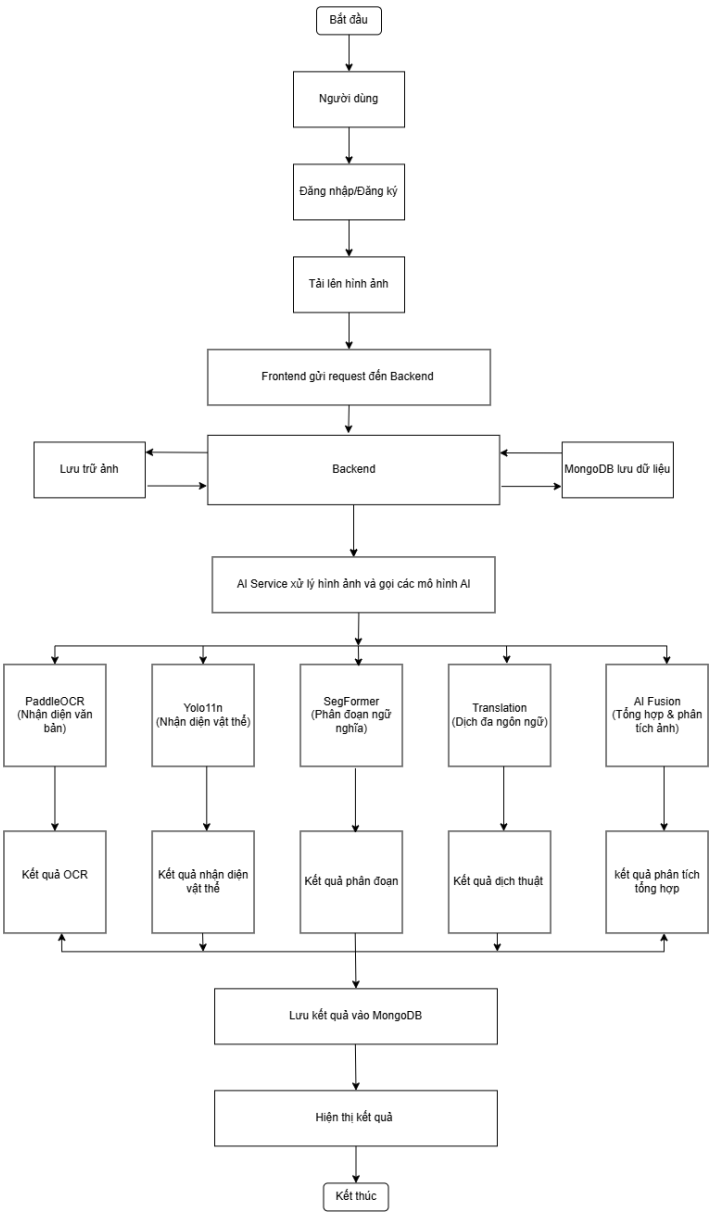
| **Tác nhân** | **Ca sử dụng** | **Điều kiện / quan hệ** |
|---|---|---|
| Người dùng | Đăng ký / đăng nhập | Cấp JWT sau khi đăng nhập |
| Người dùng | Tải ảnh lên | Cần đăng nhập |
| Người dùng | Nhận dạng văn bản (OCR) | Yêu cầu ảnh đã upload |
| Người dùng | Dịch văn bản | Có thể dùng nội dung từ kết quả OCR |
| Người dùng | Phát hiện vật thể | Yêu cầu ảnh đã upload |
| Người dùng | Phân tích ảnh đa mô hình | Yêu cầu ảnh đã upload |

| | | |
|---|---|---|
| Người dùng | Phân tích camera trực tiếp | Gửi frame riêng; có thể chuyển ảnh chụp thành ảnh tĩnh |
| Người dùng | Xem lịch sử | Xem bản ghi OCR, detection, phân tích |
| Người dùng | Xóa ảnh | Xóa asset Cloudinary và metadata |

- Mô tả: người dùng có thể đăng ký/đăng nhập, tải ảnh, thực hiện một hoặc nhiều loại phân tích và truy vấn lịch sử.

### 3.6. Sơ đồ luồng tổng thể

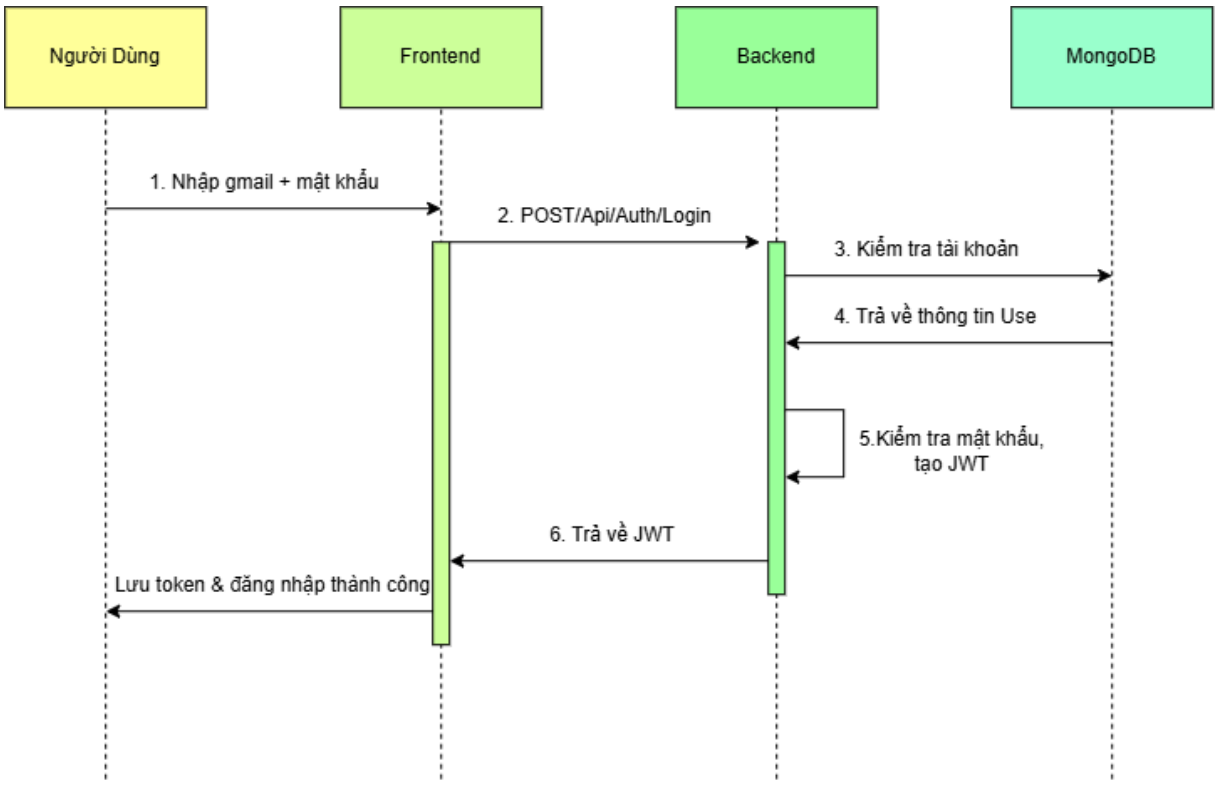
1. Bắt đầu; kiểm tra người dùng đã đăng nhập chưa. Nếu chưa, đăng nhập/đăng ký để nhận JWT.
2. Chọn chức năng, sau đó rẽ nhánh: ảnh tĩnh hoặc camera trực tiếp.
3. Nhánh ảnh tĩnh: upload ảnh lên backend → backend kiểm tra file → Cloudinary lưu ảnh → MongoDB lưu metadata → chọn OCR / detection / image analysis → backend kiểm tra quyền sở hữu → backend gọi AI service → AI trả kết quả → backend lưu bản ghi kết quả → frontend hiển thị và lưu vào history.
4. Nhánh camera trực tiếp: gửi frame tới smart-camera → AI phát hiện, hợp nhất và tracking → frontend vẽ overlay và HUD → nếu người dùng chụp ảnh để lưu thì chuyển sang nhánh ảnh tĩnh (upload), nếu không thì tiếp tục gửi frame.



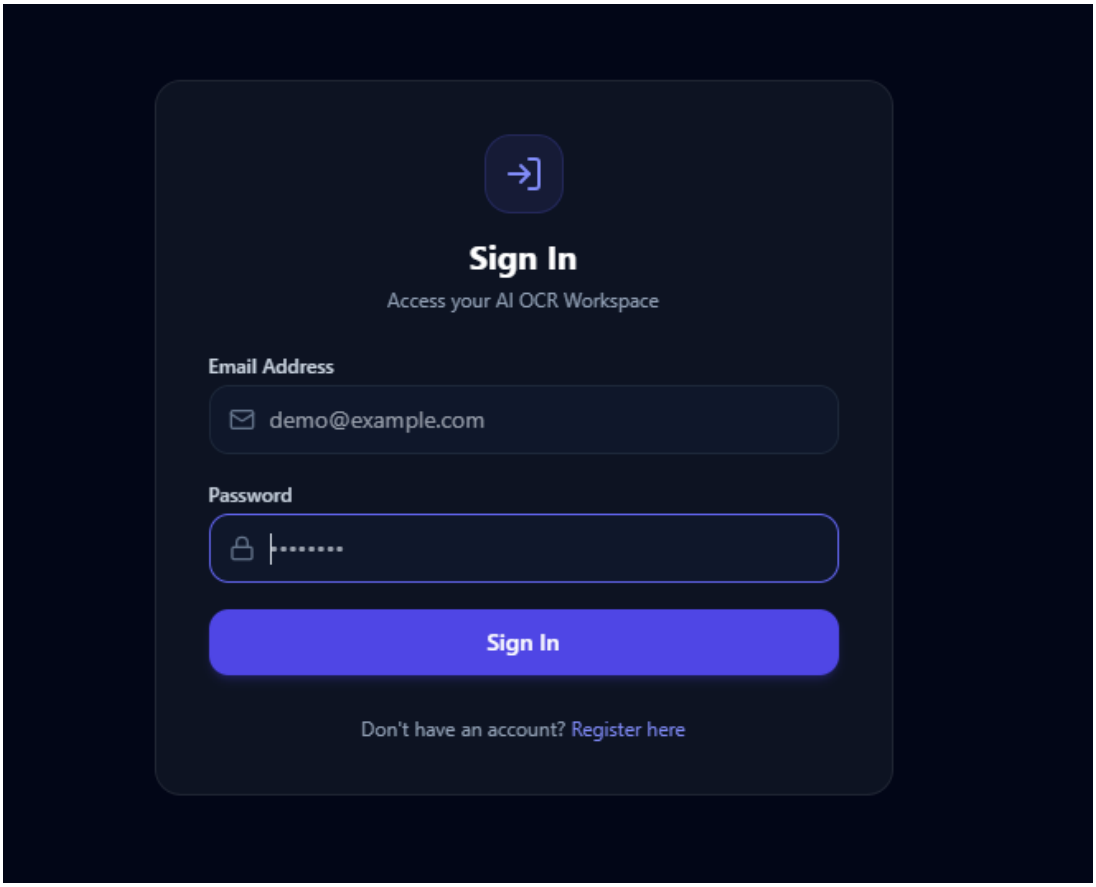

Hình 3.3. Sơ đồ luồng tổng thể hệ thống

## 3.7. Sơ đồ tuần tự đăng nhập

1. Người dùng nhập email và mật khẩu trên Frontend/AuthContext.
2. Frontend gửi POST /api/auth/login tới AuthController.
3. AuthController gọi AuthService.login(LoginDto).

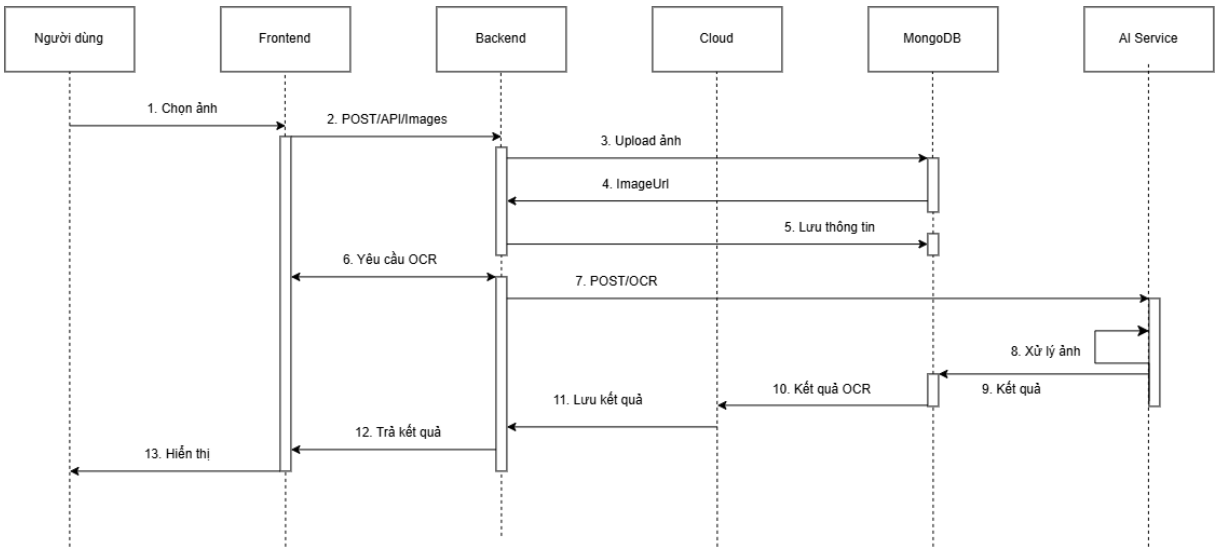



Hình 3.4. Sơ đồ tuần tự đăng nhập




Hình 3.5. Giao diện đăng nhập hệ thống

### 3.8. Sơ đồ tuần tự tải ảnh lên

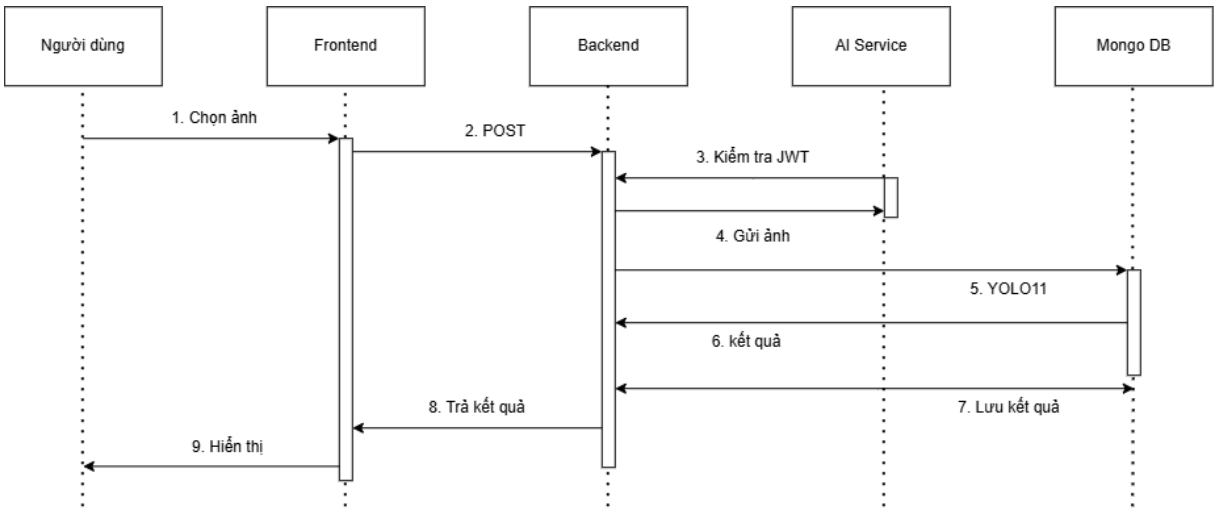



Hình 3.6. Sơ đồ tuần tự Upload ảnh

- Frontend gửi yêu cầu đến Backend, Backend lưu thông tin hình ảnh, gửi yêu cầu xử lý đến AI Service và nhận kết quả OCR. Kết quả được lưu vào MongoDB và trả về Frontend để hiển thị cho người dùng.

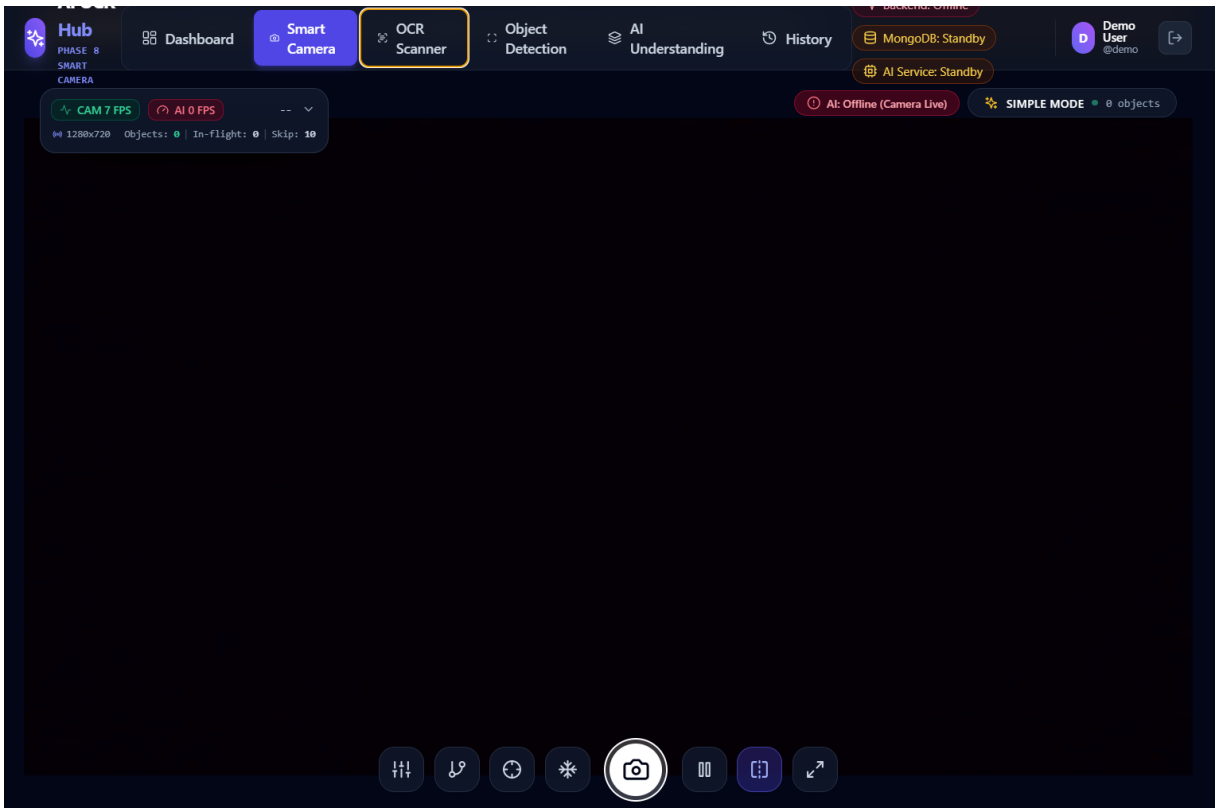
1. Người dùng chọn file ảnh trên Frontend/ImageService.
2. Frontend gửi POST /api/images (multipart, JWT) tới ImagesController.
3. Controller gọi ImagesService.uploadImage(file, userId, source, capture-dAt).
4. Service kiểm tra file, MIME và dung lượng.
5. Service upload buffer lên Cloudinary và nhận URL, publicId, kích thước.
6. Service lưu metadata và userId vào MongoDB.
7. Service trả metadata đã sanitize; controller trả imageId, imageUrl và meta-data.
8. Frontend hiển thị ảnh đã tải lên.

### 3.9. Sơ đồ tuần tự nhận diện vật thể



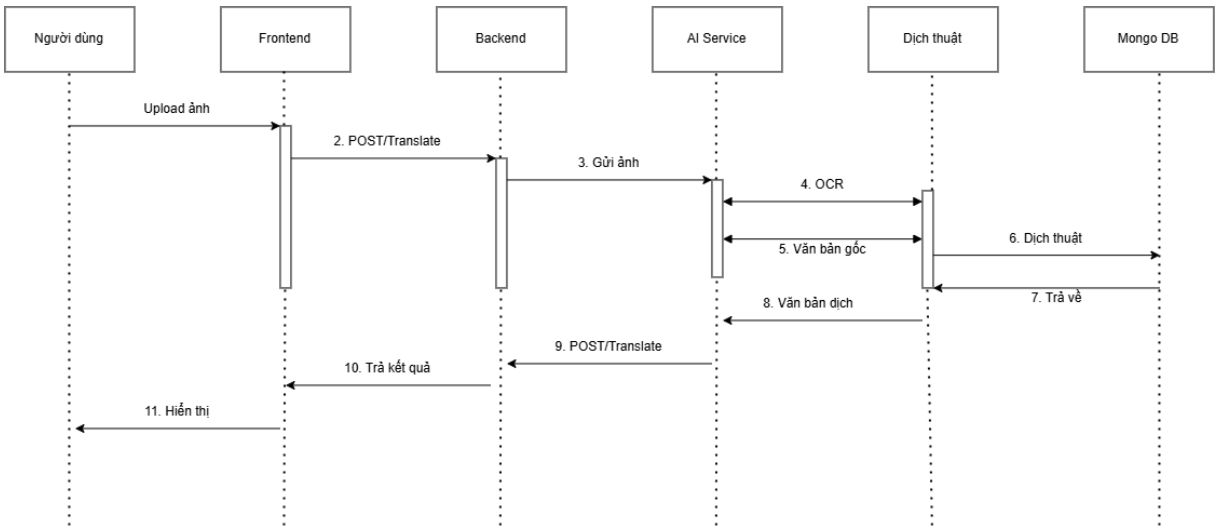

Hình 3.7. Tuần tự nhận diện vật thể

- Sơ đồ tuần tự mô tả quá trình người dùng chọn ảnh và yêu cầu nhận diện vật thể. Frontend gửi request đến Backend, Backend kiểm tra JWT và chuyển ảnh đến AI Service. AI Service sử dụng mô hình YOLO11n để nhận diện vật thể, sau đó kết quả được trả về Backend, lưu vào MongoDB và hiển thị trên Frontend.




Hình 3.8. Giao diện hình ảnh bằng Camera

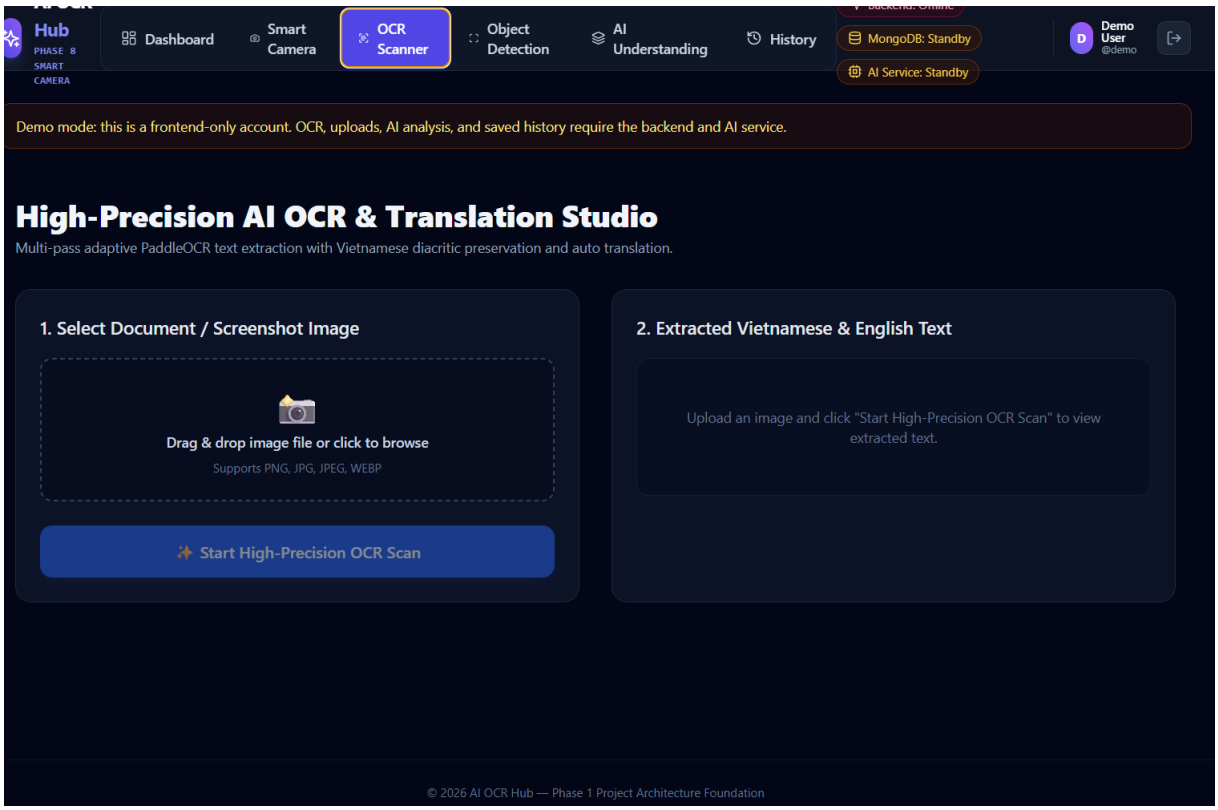
## 3.10. Sơ đồ tuần tự OCR và dịch thuật




Hình 3.9. Tuần tự OCR và dịch thuật

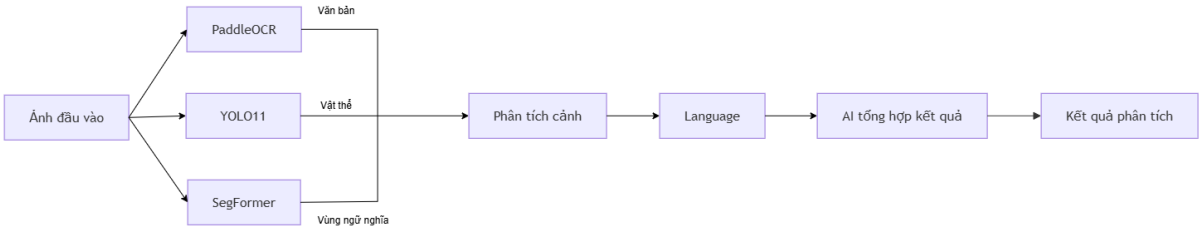
1. Người dùng chọn ảnh và chạy OCR trên OCRPage.
2. Frontend gửi POST /api/images; backend lưu metadata ảnh sau upload và trả imageId.

3. Frontend gửi POST /api/ocr {imageId} kèm JWT; backend kiểm tra quyền sở hữu ảnh.
4. Backend gửi POST /ocr {imageUrl} tới FastAPI; AI tiền xử lý và nhận dạng bằng PaddleOCR.
5. PaddleOCR trả các dòng chữ, box, confidence; AI trả fullText và OCR metadata.
6. Backend lưu OCRResult và trả kết quả OCR cho frontend.
7. Người dùng chọn ngôn ngữ và bấm dịch; frontend gửi POST /api/translation {text, sourceLanguage, targetLanguage}.
8. Backend gửi POST /translate tới AI; AI dịch từng dòng, thử translator dự phòng.
9. Backend lưu lịch sử dịch và trả kết quả; frontend hiển thị văn bản OCR và bản dịch.




Hình 3.10. Giao diện dạng dạng văn bản và dịch thuật

### 3.11. Sơ đồ hoạt động phân tích ảnh đa mô hình




Hình 3.11. Hoạt động phân tích ảnh đa mô hình

1. Nhận imageId và mode; backend kiểm tra JWT, ảnh và chủ sở hữu.
2. Backend gọi AI /image-analysis bằng imageUrl.
3. AI tải và giải mã ảnh thành bundle dùng chung.
4. Kiểm tra cache theo hash, mode và forceOcr: nếu có thì trả kết quả cache.
5. Nếu không có cache: đánh giá chất lượng ảnh → router ước lượng text và chọn module.
6. Chạy YOLO và SegFormer; sau YOLO có thể phân tích người/vật nhỏ nếu được bật, sau SegFormer là suy luận cảnh và vùng môi trường; tiếp theo là quan hệ không gian, màu sắc, khử trùng lặp.
7. Nếu router bật OCR thì chạy PaddleOCR; nếu không thì đánh dấu OCR skipped.
8. Hợp nhất kết quả, trạng thái và thời gian; AI trả response.
9. Backend lưu ImageAnalysis; frontend hiển thị kết quả và lịch sử.

- Các mode được router hỗ trợ gồm auto, fast, balanced, accurate và everything. Ví dụ, fast giảm kích thước suy luận và tắt phân tích người; balanced bật OCR khi ảnh được xem là tài liệu hoặc có tín hiệu văn bản đủ cao; everything có thể bật open-vocabulary và tiling với ảnh lớn. Các điều kiện cụ thể nằm trong ai_router_service.py; đây là heuristic, không phải phân loại ảnh đã được chứng minh chính xác cho mọi dữ liệu.

**Tiểu kết chương 3**

Thiết kế hiện tại phân tách giao diện, nghiệp vụ và inference. Luồng ảnh tĩnh tập trung vào imageId, xác thực quyền sở hữu, gọi AI bằng URL và lưu kết quả. Luồng camera gửi frame trực tiếp để giảm vòng lưu trữ. Pipeline tổng hợp có routing và trạng thái theo module để phản ánh các kết quả thành công, bỏ qua, fallback hoặc lỗi.

# CHƯƠNG 4. KIỂM THỬ CÁC CHỨC NĂNG

## 4.1. Môi trường kiểm thử

| Thành phần | Công nghệ thể hiện trong manifest/source | Cấu hình liên quan |
|---|---|---|
| Frontend | React 18, TypeScript, Vite, Axios, React Router | frontend/package.json, frontend/vite.config.ts |
| Backend | NestJS 10, Node.js, Mongoose, JWT, Axios | backend/package.json, back-end/src/config/configuration.ts |
| AI service | Python, FastAPI, Uvicorn, PaddleOCR, Ultralytics, Transformers, OpenCV, deep-translator | ai-service/requirements.txt |
| Database | MongoDB qua Mongoose | MONGODB_URI; mặc định source dùng MongoDB local |

## 4.2. Kiểm thử chức năng

| Mã | Chức năng | Dữ liệu/kịch bản trong test code | Điều kiện được kiểm tra |
|---|---|---|---|
| TC-F01 | Health check | Gọi /health và /api/health | HTTP 200; AI báo các module; backend báo trạng thái MongoDB và AI |
| TC-F02 | Đăng nhập | Tài khoản kiểm thử định nghĩa trong script | HTTP thành công và response có access token |
| TC-F03 | Upload ảnh | Multipart ảnh JPEG/PNG; một script thêm source=camera, capturedAt | Nhận image ID; metadata camera được giữ trong response |
| TC-F04 | Lịch sử ảnh | Gọi /api/images với JWT | Tìm được ảnh đã upload của tài khoản |

| Mã | Chức năng | Dữ liệu/kịch bản trong test code | Điều kiện được kiểm tra |
|---|---|---|---|
| TC-F05 | OCR / detection / image analysis history | Gọi các endpoint lịch sử | Response có các danh sách bản ghi tương ứng |
| TC-F06 | Chi tiết bản ghi | Gọi endpoint theo ID | Bản ghi được trả cho chủ sở hữu |
| TC-F07 | Kiểm tra quyền | User B thử truy cập ảnh hoặc kết quả User A | Các script kỳ vọng từ chối, thường là HTTP 403/404 |
| TC-F08 | Xóa ảnh | Giao diện xác nhận rồi gọi API delete | Frontend loại ảnh khỏi danh sách khi API báo thành công |
| TC-F09 | Smart camera | Gửi ảnh sample tới router hoặc frame qua API | Trả trạng thái, thiết bị, thời gian, detections và số track |

- Các điều kiện trong bảng mô tả nội dung có trong script; chúng chưa phải bảng kết quả chạy trong một môi trường cụ thể. Ngoài ra, controller smart-camera NestJS hiện không gắn guard JWT, nên không thể dùng các kiểm thử quyền trên các module ảnh tĩnh để kết luận route camera có cùng mức xác thực.

### 4.3. Kiểm thử OCR

| Ca kiểm thử | Đầu vào | Chỉ số/đầu ra cần ghi nhận |
|---|---|---|
| OCR hóa đơn | doc_invoice.png | Số dòng, văn bản nhận dạng, confidence từng dòng và confidence trung bình |
| Ảnh văn bản nhiều dòng | text_heavy.png hoặc receipt.jpg trong bộ ảnh mẫu | Thứ tự dòng, độ đầy đủ văn bản, xuống dòng |

| | | |
|---|---|---|
| Ảnh chữ chất lượng thấp | low_text.jpg | Có chạy lượt contrast hay không; isUncertain; confidence |
| OCR qua API AI | URL ảnh Cloudinary | HTTP status, processingTimeMs, lines |
| Lịch sử và phân quyền | Kết quả OCR User A; truy cập bằng User B | HTTP response và dữ liệu không bị lộ cho user khác |

- OCRService chọn lượt thứ hai theo điều kiện confidence và có thể giữ lượt đầu nếu xử lý lượt hai lỗi. Vì vậy, một lần OCR thành công không chứng minh mọi ký tự được nhận dạng đúng. Cần đối chiếu văn bản trả về với ground truth cho từng ảnh nếu muốn báo cáo độ chính xác định lượng.

### 4.4. Kiểm thử dịch thuật

test_audit_translation.py có các luồng:

1. Dịch tiếng Anh sang tiếng Việt với văn bản nhiều dòng và kiểm tra sự hiện diện của ký tự xuống dòng trong output.
2. Dịch tiếng Việt sang tiếng Anh.
3. Gọi trực tiếp AI service với sourceLanguage=’auto’.
4. Lấy lịch sử dịch của User A.
5. Đăng nhập User B và kiểm tra lịch sử độc lập.

### 4.5. Kiểm thử hệ thống đa mô hình

| **Ca** | **Ảnh mẫu trong repository** | **Module cần xem** | **Kết quả cần ghi** |
|---|---|---|---|
| Phong cảnh | beach.jpg, street.jpg | YOLO, SegFormer, scene inference, vision/fallback | Module status, object count, top regions, mô tả, tổng thời gian |
| Văn bản/hóa đơn | doc_invoice.png, receipt.jpg, text_heavy.png | OCR, routing, translation nếu được gọi | Text, line count, confidence, quyết định bật/tắt OCR |

| Ca | Ảnh mẫu trong repository | Module cần xem | Kết quả cần ghi |
|---|---|---|---|
| Ảnh người và phụ kiện | person_phone.jpg, ảnh trong testim-ages/accessories/ | YOLO, person analysis, open-vocabulary, hierarchy | Nhãn, confidence, parent-child, loại phụ kiện |
| Ảnh chất lượng thấp | low_text.jpg, object_poor.jpg | Quality assessment, OCR, detection | Warnings, module status, confidence và các lỗi |
| Camera trực tiếp | Frame từ webcam hoặc sample frame | YOLO, pose, OCR tùy chọn, fusion, tracking | Frame ID, trạng thái, detections, track count, timing |
| URL ảnh không hợp lệ | URL không truy cập/không phải ảnh | Input validation và xử lý lỗi | HTTP status và nội dung lỗi |

- Pipeline có thể trả partial_success khi module lỗi và ghi trạng thái theo module. Vision-language có thể trả isFallback=true nếu Gemini không khả dụng hoặc bộ tổng hợp cục bộ được dùng. Khi đánh giá cần lưu rõ trạng thái này, không chỉ chụp phần mô tả cuối.

### 4.7. Đánh giá kết quả

Kết quả cần đo khi chạy thực tế

| STT | Chức năng | Kết quả |
|---|---|---|
| 1 | Đăng nhập | PASS |
| 2 | Tải ảnh | PASS |
| 3 | Mở camera | PASS |
| 4 | Dịch thuật | PASS |

**Tiểu kết chương 4**

Chương 4 hệ thống hóa các kịch bản kiểm thử có trong repository và chỉ rõ những chỉ số cần đo trong lần chạy thực tế. Các điều kiện trong script chỉ có giá trị bằng chứng khi đi kèm log và kết quả chạy được ghi lại đúng môi trường.

# CHƯƠNG 5. HƯỚNG PHÁT TRIỂN VÀ KẾT LUẬN

## 5.1. Tổng kết kết quả đạt được

1. Ứng dụng web nhiều tầng: frontend React/TypeScript, backend NestJS và AI service FastAPI có cấu hình giao tiếp qua HTTP.
2. Tài khoản và phân vùng dữ liệu: đăng ký, đăng nhập, băm mật khẩu bằng bcrypt, cấp JWT và truy vấn dữ liệu theo userId.
3. Upload và quản lý ảnh: kiểm tra loại MIME/dung lượng, upload Cloudinary, lưu metadata MongoDB, liệt kê, xem và xóa ảnh.
4. OCR: tích hợp PaddleOCR tiếng Việt, hỗ trợ nhiều cấu trúc output, sắp xếp dòng theo vị trí, hai lượt xử lý có điều kiện, hậu xử lý và trường confidence.
5. Dịch: hỗ trợ truyền ngôn ngữ nguồn/đích, tự chọn vi/en theo heuristic, dịch từng dòng qua hai translator và lưu lịch sử.
6. Object detection: gọi YOLO, xuất nhãn/confidence/box/kích thước ảnh và lưu kết quả.
7. Phân đoạn: dùng SegFormer để tạo thống kê vùng theo lớp và diện tích, có ánh xạ nhãn về các nhóm môi trường.
8. Phân tích đa mô hình: dùng quality assessment, heuristic routing, YOLO, segmentation, OCR có điều kiện, phân tích người/vật nhỏ, quan hệ không gian, suy luận cảnh, vision-language và fusion.
9. Camera thời gian thực: nhận frame multipart/Base64, có chế độ simple/detailed/everything ở router, detection fusion, hierarchy, tracking, coordinate transform và timing.
10. Kiểm thử/huấn luyện: có script tích hợp API, benchmark detector, calibration threshold, export/evaluate và huấn luyện detector ứng viên.

- Các kết quả trên là kết quả về mặt cấu trúc chức năng được mã hóa. Việc các tính năng chạy ổn định, đạt chỉ số định lượng cụ thể hoặc triển khai

production cần được xác nhận bằng chạy thử trên môi trường cấu hình đầy đủ.

## 5.2. Những hạn chế của hệ thống

Hạn chế về kết quả AI:

- Chất lượng OCR còn phụ thuộc độ rõ, độ nghiêng, độ tương phản và bố cục ảnh. Logic confidence và lượt OCR thứ hai không thay thế đánh giá bằng ground truth.
- Tự nhận dạng ngôn ngữ trong translation service chỉ dùng dấu tiếng Việt để phân biệt vi/en; các ngôn ngữ khác hoặc văn bản ngắn có thể bị phân loại sai.
- Dịch thuật dựa vào translator bên ngoài; nếu cả hai adapter lỗi, service trả lại dòng gốc làm fallback. Cần hiển thị rõ fallback để người dùng không hiểu nhầm là bản dịch thành công.
- SegFormer trả thống kê nhãn và phần trăm vùng, không trả mask pixel trong response hiện tại; nhãn vùng nhỏ hơn 1% bị loại.
- Các model phụ thuộc file trọng số và thư viện runtime. Ví dụ YOLO-World chỉ được dùng trong smart camera khi trọng số tồn tại; model candidate trong pipeline huấn luyện không tự động chứng minh đã được triển khai.

Hạn chế về đánh giá

- Kịch bản audit cần backend, AI service, MongoDB, Cloudinary, tài khoản và ảnh sẵn có. Một số script hard-code đường dẫn D:/hocWeb/P_AI, nên cần thay cấu hình đường dẫn trước khi chạy trên máy khác.
- Script integration thể hiện các assertion và trường cần kiểm tra, nhưng source script đơn thuần không cho biết lần chạy cụ thể nào đã pass.
- Độ chính xác OCR, detection, dịch, latency và tỷ lệ lỗi tổng thể chưa thể kết luận chỉ từ cấu hình và thuật toán.

Hạn chế về vận hành và tích hợp

- Backend phụ thuộc nhiều biến môi trường như MongoDB URI, JWT secret, Cloudinary credentials, URL frontend/AI và kích thước upload.
- Smart-camera controller trong backend không khai báo JwtAuthGuard, khác với các controller ảnh tĩnh; cần rà soát và thống nhất chính sách xác thực trước khi triển khai.
- Camera gửi request liên tục và giao diện có logic chống request đồng thời/bỏ qua frame; chất lượng trải nghiệm vẫn cần đo trên thiết bị, mạng và tải đồng thời thực tế.

### 5.3. Hướng phát triển trong tương lai

1. Đo định lượng đúng chuẩn: bổ sung ground truth rõ ràng cho OCR, đánh giá detection bằng metric chuẩn, tách validation/benchmark và lưu log theo phiên bản model.
2. Làm rõ trạng thái fallback: trả trường thể hiện rõ dịch vụ dịch đã dùng adapter nào, dòng nào giữ nguyên do lỗi; giao diện cảnh báo rõ kết quả dịch fallback.
3. Tăng khả năng diễn giải phân đoạn: nếu nhu cầu giao diện cần xem mask, bổ sung mask/overlay hoặc endpoint ảnh mask thay vì chỉ danh sách vùng.
4. Bảo vệ API nhất quán: rà soát guard cho smart camera, status và reset tracker; áp dụng xác thực/giới hạn request phù hợp với mục tiêu sử dụng.
5. Theo dõi hiệu năng camera: thu thập latency theo từng chặng và tỷ lệ frame bỏ qua trên thiết bị mục tiêu; tối ưu model hoặc kích thước ảnh dựa trên số liệu thực tế.

# KẾT LUẬN

Đồ án "Xây dựng hệ thống nhận diện vật thể và dịch nội dung hình ảnh đa ngôn ngữ" được thực hiện nhằm nghiên cứu và xây dựng một ứng dụng có khả năng khai thác thông tin từ hình ảnh thông qua việc tích hợp nhiều công nghệ trí tuệ nhân tạo. Nội dung đồ án đã trình bày từ cơ sở lý thuyết, lựa chọn công nghệ, phân tích yêu cầu, thiết kế kiến trúc cho đến xây dựng các kịch bản kiểm thử và đề xuất hướng phát triển hệ thống.

Về mặt kỹ thuật, hệ thống được tổ chức thành ba thành phần chính: giao diện người dùng sử dụng React và TypeScript, backend nghiệp vụ sử dụng NestJS và dịch vụ xử lý AI sử dụng FastAPI cùng Python. Các thành phần được thiết kế để phối hợp thông qua API, trong đó backend đảm nhiệm xác thực, quản lý dữ liệu, kiểm tra quyền truy cập và lưu trữ lịch sử; dịch vụ AI thực hiện các tác vụ xử lý hình ảnh chuyên biệt.

Các chức năng được thể hiện trong mã nguồn bao gồm quản lý tài khoản, tải và quản lý hình ảnh, nhận dạng văn bản bằng PaddleOCR, dịch văn bản, phát hiện vật thể bằng YOLO, phân đoạn ngữ nghĩa bằng SegFormer, phân tích ảnh đa mô hình và xử lý khung hình camera trực tiếp. Việc tổ chức các chức năng thành những dịch vụ riêng biệt giúp hệ thống có cấu trúc rõ ràng, thuận lợi cho việc bảo trì, mở rộng và thay thế mô hình khi có nhu cầu.

Tuy nhiên, hệ thống vẫn còn những hạn chế cần tiếp tục khắc phục. Chất lượng kết quả phụ thuộc vào dữ liệu đầu vào, mô hình sử dụng, cấu hình môi trường và khả năng hoạt động của các dịch vụ bên ngoài. Bên cạnh đó, việc đánh giá độ chính xác, thời gian xử lý, khả năng chịu tải và mức độ an toàn của toàn hệ thống cần được thực hiện bằng các thử nghiệm thực tế có dữ liệu, nhật ký và tiêu chí đo lường cụ thể. Vì vậy, những chức năng được mô tả trong báo cáo cần được phân biệt với các kết quả thực nghiệm đã được xác nhận.

# TÀI LIỆU THAM KHẢO

1. GitHub
2. Giáo án Đồ Án Trí Tuệ Nhân Tạo
3. Có sử dụng các chat AI